Вопросы к экзамену по дисциплине

Конструирование алгоритмов и структур данных

Лекция 1. Введение. Асимптотический анализ. Сортировки.

1. Дайте определение асимптотической нотации O, Ω, Θ. В чём разница между ними? Проанализируйте время работы алгоритма поиска минимума в неотсортированном массиве. Почему нижняя оценка для этой задачи – ( ) Ω n ?

Определения асимптотических нотаций

- **O(g(n))** — *асимптотическая верхняя оценка*:
  $f(n) = O\big(g(n)\big)$, если $\exists c > 0, \exists n_0 : \forall n \geq n_0 \Rightarrow 0 \leq f(n) \leq c \cdot g(n)$.
- **Ω(g(n))** — *асимптотическая нижняя оценка*:
  $f(n) = \Omega\big(g(n)\big)$, если $\exists c > 0, \exists n_0 : \forall n \geq n_0 \Rightarrow 0 \leq c \cdot g(n) \leq f(n)$.
- **Θ(g(n))** — *асимптотически точная оценка* (и сверху, и снизу):
  $f(n) = \Theta\big(g(n)\big)$, если $\exists c_1 > 0, c_2 > 0, n_0 : \forall n \geq n_0 \Rightarrow 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$.

Разница

- O — “не быстрее чем…”, верхняя граница.
- Ω — “не медленнее чем…”, нижняя граница.
- Θ — “растёт как…”, точный порядок роста (эквивалентно O и Ω одновременно).

Поиск минимума в неотсортированном массиве (время работы)

По лекции для алгоритма поиска минимума получается линейное время: $T(n) = 2n \Rightarrow O(n)$.

C# (перевод псевдокода find_min):

csharp

```csharp
static int FindMin(int[] arr)
{
    int minVal = arr[0];
    for (int i = 1; i < arr.Length; i++)
    {
        int num = arr[i];
        if (num < minVal)
            minVal = num;
    }
    return minVal;
}
```

Почему нижняя оценка для задачи — $\Omega(n)$?
Потому что для корректного поиска минимума в **неотсортированном** массиве алгоритм обязан “посмотреть” каждый элемент хотя бы один раз, иначе можно пропустить истинный минимум. Следовательно, любой алгоритм имеет нижнюю границу $\Omega(n)$, а данный — $\Theta(n)$.

2. Опишите алгоритм сортировки вставками. Какова его временная сложность в худшем, лучшем и среднем случае? Что такое «инверсия в массиве»? Опишите модификацию сортировки слиянием, позволяющую подсчитать количество инверсий за (O · log n n)

Сортировка вставками

Алгоритм: Массив условно делится на отсортированную и неотсортированную части. Каждый следующий элемент вставляется на своё место в отсортированной части массива путём сдвига элементов вправо.

Временная сложность:

- лучший случай (массив отсортирован): **O(n)**
- средний случай: **O(n²)**
- худший случай (обратный порядок): **O(n²)**

Код сортировки вставками (C#):

csharp

```
static void InsertionSort(int[] a)
{
    for (int i = 1; i < a.Length; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Инверсия в массиве

Инверсия — это пара индексов (i < j), для которых выполняется (a[i] > a[j]). Количество инверсий показывает степень неупорядоченности массива.

Подсчёт инверсий за O(n·log n) (модификация сортировки слиянием)

Идея: Используется сортировка слиянием. При слиянии двух отсортированных подмассивов, если элемент из правой части меньше элемента из левой, то он образует инверсии со всеми оставшимися элементами левой части.

Сложность алгоритма: **O(n·log n)**.

Код подсчёта инверсий (C#):

csharp

```
static long CountInversions(int[] a)
{
    int[] temp = new int[a.Length];
    return MergeSort(a, temp, 0, a.Length - 1);
}

static long MergeSort(int[] a, int[] temp, int left, int right)
{
    long invCount = 0;
    if (left < right)
    {
        int mid = (left + right) / 2;
        invCount += MergeSort(a, temp, left, mid);
        invCount += MergeSort(a, temp, mid + 1, right);
        invCount += Merge(a, temp, left, mid, right);
    }
    return invCount;
}

static long Merge(int[] a, int[] temp, int left, int mid, int right)
{
    int i = left, j = mid + 1, k = left;
    long invCount = 0;

    while (i <= mid && j <= right)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
        {
            temp[k++] = a[j++];
            invCount += (mid - i + 1);
        }
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (int p = left; p <= right; p++)
        a[p] = temp[p];

    return invCount;
}
```

3. Опишите алгоритм сортировки слиянием. Почему его сложность всегда (O · log n n) ?

Сортировка слиянием

Алгоритм: Массив рекурсивно делится на две части до подмассивов длины 1. Затем подмассивы попарно сливаются в отсортированном порядке.

Этапы:

1. Разделить массив на две половины.
2. Отсортировать каждую половину рекурсивно.
3. Слить две отсортированные половины в один массив.

Почему сложность всегда O(n·log n)

- Глубина рекурсии равна **log n** (массив каждый раз делится пополам).
- На каждом уровне рекурсии выполняется слияние всех элементов, что требует **O(n)** операций.
- Итоговая сложность: **O(n·log n)** независимо от входных данных.

Код сортировки слиянием (C#)

csharp

```
static void MergeSort(int[] a)
{
    int[] temp = new int[a.Length];
    MergeSortRec(a, temp, 0, a.Length - 1);
}

static void MergeSortRec(int[] a, int[] temp, int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;
        MergeSortRec(a, temp, left, mid);
        MergeSortRec(a, temp, mid + 1, right);
        Merge(a, temp, left, mid, right);
    }
}

static void Merge(int[] a, int[] temp, int left, int mid, int right)
{
    int i = left, j = mid + 1, k = left;
```

```
    while (i <= mid && j <= right)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (int p = left; p <= right; p++)
        a[p] = temp[p];
}
```

Мастер-теорема

Рассматривается рекуррентное соотношение вида
$T(n) = a\,T(n/b) + f(n)$, где $a \geq 1$, $b > 1$.

1. Если $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$, то
$$T(n) = \Theta\left(n^{\log_b a}\right)$$

2. Если $f(n) = \Theta\left(n^{\log_b a}\right)$, то
$$T(n) = \Theta\left(n^{\log_b a}\log n\right)$$

3. Если $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ и выполнено условие регулярности, то
$T(n) = \Theta\left(f(n)\right)$

Применение к сортировке слиянием

Для сортировки слиянием:

- $a = 2$
- $b = 2$
- $f(n) = O(n)$

$\log_b a = \log_2 2 = 1$, следовательно
$$f(n) = \Theta\left(n^{\log_b a}\right)$$

По второму случаю мастер-теоремы:
$T(n) = \Theta(n\log n)$

Применение к бинарному поиску

Для бинарного поиска:

- $a = 1$
- $b = 2$
- $f(n) = O(1)$

$\log_b a = \log_2 1 = 0$, следовательно

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

По второму случаю мастер-теоремы:
$T(n) = \Theta(\log n)$

Лекция 2. Бинарная куча. Пирамидальная сортировка.

4. Что такое «очередь с приоритетом»? Какие основные операции она должна поддерживать? Опишите структуру данных «бинарная куча» для минимума. Какое свойство она должна поддерживать?

Очередь с приоритетом — АТД, который хранит *мультимножество* сравнимых элементов и поддерживает основные операции:

- `insert(x)` — добавить элемент
- `get_min()` (или `find_min/peek`) — вернуть минимальный элемент
- `remove_min()` (или `extract_min`) — удалить минимальный элемент (иногда `extract_min` совмещает «вернуть + удалить»)

Бинарная куча (min-heap) как реализация:

- Это *полное бинарное дерево* (последний уровень заполняется слева направо).
- Хранится в массиве: корень `a[0]`.
  - `left(i) = 2*i + 1`
  - `right(i) = 2*i + 2`
  - `parent(i) = (i - 1) / 2` (целочисленно)

Свойство (для минимума): значение в каждом узле **≤** значений в его дочерних узлах; следствие — **минимум всегда в корне**.

Операции очереди с приоритетом (min): `insert(x)`, `get_min()`, `remove_min()` (иногда объединяют в `extract_min`). filecite turn1file12

Бинарная куча (min-heap): полное бинарное дерево, хранится в массиве: `left(i)=2*i+1`, `right(i)=2*i+2`, `parent(i)=(i-1)//2`. Свойство: в каждом узле значение ≤ значений в детях, поэтому минимум в корне. filecite turn1file17

Реализация на C# (`heap_get_min`, `heap_insert`, `heap_remove_min`, `sift_up`, `sift_down`)

csharp

```
using System;
using System.Collections.Generic;
```

```
public class MinHeap
{
    private readonly List<int> a = new List<int>();

    public int Count => a.Count;

    public int GetMin()
    {
        if (a.Count == 0) throw new InvalidOperationException("Куча пуста");
        return a[0];
    }

    public void Insert(int x)
    {
        a.Add(x);
        SiftUp(a.Count - 1);
    }

    public int RemoveMin()
    {
        if (a.Count == 0) throw new InvalidOperationException("Куча пуста");

        int minVal = a[0];
        int lastVal = a[a.Count - 1];
        a.RemoveAt(a.Count - 1);

        if (a.Count > 0)
        {
            a[0] = lastVal;
            SiftDown(0);
        }

        return minVal;
    }

    private void SiftUp(int idx)
    {
        while (idx > 0)
        {
            int parentIdx = (idx - 1) / 2;
            if (a[idx] < a[parentIdx])
            {
                (a[idx], a[parentIdx]) = (a[parentIdx], a[idx]);
                idx = parentIdx;
            }
            else
            {
                break;
            }
        }
    }

    private void SiftDown(int idx)
    {
        int n = a.Count;
```

```
        while (idx * 2 + 1 < n)
        {
            int leftIdx = 2 * idx + 1;
            int rightIdx = 2 * idx + 2;

            int minChildIdx = leftIdx;
            if (rightIdx < n && a[rightIdx] < a[leftIdx])
                minChildIdx = rightIdx;

            if (a[idx] <= a[minChildIdx])
                break;

            (a[idx], a[minChildIdx]) = (a[minChildIdx], a[idx]);
            idx = minChildIdx;
        }
    }
}
```

5. Объясните алгоритмы операций вставка и извлечения минимума в бинарной куче. Почему их время работы – (O logn) ? Как работает алгоритм построения кучи из неупорядоченного массива? Почему его сложность – ( ) O n , а не (O · log n n) ?

Вставка в min-heap (insert) и почему **O(log n)**

Алгоритм:

1. Добавляем новый элемент x **в конец массива** (в последнюю свободную позицию).
2. Делаем `sift_up` (просеивание вверх): сравниваем с родителем, если меньше — меняем местами, повторяем до корня или пока порядок не восстановится.

Псевдокод:

```
heap_insert(a, x)
    a.append(x)
    sift_up(a, len(a) - 1)

sift_up(a, idx)
    while idx > 0
        parent_idx = (idx - 1) // 2
        if a[idx] < a[parent_idx]
            swap(a[idx], a[parent_idx])
            idx = parent_idx
        else
            break
```

Почему O(log n): **высота дерева ~ log2 n**, `sift_up` поднимает элемент не более чем на высоту дерева.

Извлечение минимума (remove_min) и почему **O(log n)**

Алгоритм:

1. Запоминаем корень (минимум).
2. Перемещаем **последний** элемент массива в корень.
3. Делаем `sift_down` (просеивание вниз): сравниваем с детьми, если больше — меняем местами с **наименьшим** ребёнком, повторяем пока порядок не восстановится или не дошли до листа.

Псевдокод:

```
heap_remove_min(a)
    if not a: raise Exception("Куча пуста")
    min_val = a[0]
    last_val = a.pop()
    if a:
        a[0] = last_val
        sift_down(a, 0)
    return min_val, a

sift_down(a, idx)
    n = len(a)
    while idx * 2 + 1 < n
        left_idx = 2 * idx + 1
        right_idx = 2 * idx + 2
        min_child_idx = left_idx
        if right_idx < n and a[right_idx] < a[left_idx]
            min_child_idx = right_idx
        if a[idx] <= a[min_child_idx]
            break
        else
            swap(a[idx], a[min_child_idx])
            idx = min_child_idx
```

Почему O(log n): `sift_down` опускает элемент не более чем на высоту дерева, то есть **O(log n)**.

Построение кучи из неупорядоченного массива (heapify)

В файле указано 2 способа:

1. **Последовательная вставка**: вызвать `insert` для каждого из n элементов → **O(n log n)**.
2. **Линейный алгоритм**: идти **снизу вверх** и вызывать `sift_down` для каждого узла, начиная с последнего нелистового → **O(n)**.

Псевдокод (heapify для max-heap в лекциях):

```
heapify(arr)
    n = len(arr)
```

```
    start_idx = n // 2 - 1
    for i in range(start_idx, -1, -1)
        sift_down_max(arr, i, n)
```

Почему heapify — **O(n)**, а не **O(n log n)**

Обоснование из файла:

- На нижнем уровне ~ n/2 узлов, их высота 0 → `sift_down` = 0 операций
- На уровне выше ~ n/4 узлов, высота 1 → не более 1 операции
- Далее: n/8 узлов высоты 2, …, в корне 1 узел высоты h

Суммарное время:

```
T(n) = 0*(n/2) + 1*(n/4) + 2*(n/8) + 3*(n/16) + ... + h*1
```

Эта сумма ограничена сверху числом 2n, значит `T(n) = O(n)`.

6. Опишите алгоритм пирамидальной сортировки. Докажите, что его сложность равна (O · log n n) .

Пирамидальная сортировка (Heap Sort)

Идея алгоритма:
Используется бинарная куча (обычно **max-heap**). Сначала из массива строится куча, затем максимальные элементы по одному извлекаются и помещаются в конец массива.

Алгоритм (псевдокод из файла)

Построение кучи

```
heapify(a, n)
    for i ← n/2 – 1 downto 0
        sift_down(a, i, n)
```

Сортировка

```
heap_sort(a)
    n ← length(a)
    heapify(a, n)
    for i ← n – 1 downto 1
        swap(a[0], a[i])
        sift_down(a, 0, i)
```

Просеивание вниз

```
sift_down(a, i, size)
    while left(i) < size
        j ← left(i)
        if right(i) < size and a[right(i)] > a[j]
            j ← right(i)
        if a[i] ≥ a[j]
            break
```

```
        swap(a[i], a[j])
        i ← j
```

Почему сложность **O(n·log n)**

Обоснование из файла:

1. **Построение кучи** — `O(n)`
2. Основной цикл выполняется n раз
3. На каждом шаге вызывается `sift_down`, работающий за `O(log n)`

Итоговая сложность:

```
O(n) + n · O(log n) = O(n · log n)
```

Лекция 3. Бинарный поиск. Порядковые статистики.

7. Опишите алгоритм бинарного поиска в отсортированном массиве. Какова его сложность? Что такое нижняя граница и верхняя граница элемента? Как их реализовать с помощью бинарного поиска?

Бинарный поиск в отсортированном массиве

Алгоритм:
Работает только для **отсортированного массива**. На каждом шаге сравнивает искомый элемент с серединой и отбрасывает половину массива.

Псевдокод бинарного поиска

```
binary_search(a, x)
    l ← 0
    r ← n – 1
    while l ≤ r
        m ← (l + r) div 2
        if a[m] = x
            return m
        else if a[m] < x
            l ← m + 1
        else
            r ← m – 1
    return –1
```

Сложность:
На каждом шаге диапазон уменьшается в 2 раза ⇒ **O(log n)**.

Нижняя и верхняя границы

Нижняя граница (lower bound):
Индекс **первого элемента ≥ x**.

Псевдокод

```
lower_bound(a, x)
    l ← 0
    r ← n
    while l < r
        m ← (l + r) div 2
```

```
        if a[m] < x
            l ← m + 1
        else
            r ← m
    return l
```

Верхняя граница (upper bound):
Индекс **первого элемента > x**.

Псевдокод

```
upper_bound(a, x)
    l ← 0
    r ← n
    while l < r
        m ← (l + r) div 2
        if a[m] ≤ x
            l ← m + 1
        else
            r ← m
    return l
```

Сложность lower_bound и upper_bound:
Так же, как у бинарного поиска — **O(log n)**.

8. В чём заключается идея «бинарного поиска по ответу»? Приведите пример задачи, где он применяется.

Идея «бинарного поиска по ответу»

Суть:
Бинарный поиск применяется **не к массиву**, а к **множеству возможных ответов**.
Предполагается, что для ответа существует **монотонное предикатное условие**:
если для значения x условие истинно, то для всех x' > x (или < x) оно тоже истинно.

Общая схема (псевдокод из лекции)

```
binary_search_answer(l, r)
    while l < r
        m ← (l + r) div 2
        if check(m) = true
            r ← m
        else
            l ← m + 1
    return l
```

где `check(m)` — проверка, можно ли получить ответ не хуже `m`.

Пример задачи (из лекции)

Задача:
Дано n копий и два ксерокса, которые печатают за x и y секунд.
Найти **минимальное время**, за которое можно сделать n копий.

Идея применения:

- Ответ — это **время**

- Проверка `check(t)`: можно ли за `t` секунд напечатать ≥ n копий
- Функция `check(t)` монотонна ⇒ применим бинарный поиск по ответу

Почему работает

Из файла:

- множество ответов упорядочено
- проверка «подходит / не подходит» монотонна
- бинарный поиск находит **минимальный подходящий ответ**

Сложность: `O(log R · cost(check))`, где R — диапазон возможных ответов.

9. Опишите рандомизированный алгоритм быстрого выбора для нахождения k-й порядковой статистики. Какова его средняя временная сложность? Опишите идею детерминированного алгоритма медиана медиан. Почему его время работы в худшем случае – ( )n O ?

Рандомизированный быстрый выбор (Quickselect) для k-й порядковой статистики

Задача: найти k-ю порядковую статистику (элемент, который стоял бы на позиции k (с 0) в отсортированном массиве).

Идея Quickselect (Хоара): очень похож на быструю сортировку, но рекурсивно обрабатывает **только ту часть**, где находится ответ.

1. выбрать случайный опорный элемент x;
2. сделать `partition` на `< x` и `≥ x`;
3. сравнить k с позицией опорного (или границей) и уйти в нужную часть.

Псевдокод (как в лекциях, в стиле реализации):

```
quickselect(arr, k, left, right)
    if left == right
        return arr[left]

    pivot_idx = random(left, right)
    swap(arr[pivot_idx], arr[right])
    pivot_value = arr[right]

    p = partition(arr, left, right, pivot_value)

    if k == p
        return arr[p]
    else if k < p
        return quickselect(arr, k, left, p - 1)
    else
```

```
        return quickselect(arr, k, p + 1, right)
```

Средняя (ожидаемая) сложность: анализ аналогичен быстрой сортировке; в среднем на каждом шаге размер задачи уменьшается примерно вдвое **O(n) в среднем**.

Детерминированный алгоритм “медиана медиан” (BFPRT): идея и почему худший случай O(n)

Идея: выбирать опорный элемент x **не случайно**, а детерминированно так, чтобы он гарантированно делил массив не хуже, чем **3:7**.

Шаги алгоритма (из лекций):

1. разбить массив на группы по 5 (последняя может быть меньше);
2. найти медиану в каждой группе (для 5 элементов — константно);
3. рекурсивно найти медиану x среди этих медиан (медиану медиан) — это опорный;
4. разделить массив вокруг x (`partition`), получить границу `m`;
5. как в Quickselect, рекурсивно продолжить поиск в нужной части по k и `m`.

Почему время в худшем случае O(n): из свойства медианы медиан следует, что после разбиения вокруг x большая часть имеет размер не более `7n/10`.
Получается рекуррентность:

$$T(n) \leq T(n/5) + T(7n/10) + O(n)$$

В лекциях доказывается по индукции, что из этой рекуррентности следует **T(n)=O(n)** (выбор константы, например $c \geq 10a$).

Вывод: BFPRT находит k-ю порядковую статистику за **O(n) в худшем случае** (хотя константа большая, и на практике часто быстрее Quickselect).

Лекция 4. Амортизированный анализ. Стек, очередь, дек.

10. Что такое «амортизированная сложность»? В чём её отличие от сложности в худшем случае? Объясните на примере динамического массива методы агрегирования, бухгалтерского учёта и потенциалов для амортизационного анализа.

Амортизированная сложность

Определение (из файла):
Амортизированная сложность — это **средняя стоимость операции на последовательности операций**, гарантирующая верхнюю оценку на *любой префикс* этой последовательности, а не на одну операцию.

Отличие от худшего случая:

- **Худший случай** — максимальное время *одной* операции.
- **Амортизированная сложность** — средняя стоимость операции **на длинной серии операций**, даже если отдельные операции бывают дорогими.

Пример: динамический массив

Массив увеличивает размер в 2 раза при переполнении.
Операция `push_back`:

- обычно — O(1)
- иногда — O(n) (при перераспределении и копировании)

Метод агрегирования

Идея:
Считаем суммарную стоимость n операций и делим на n.

Для n операций `push_back`:

- копирование элементов при расширениях суммарно ≤ 2n
- всего операций ≤ 3n

Амортизированная стоимость:

```
3n / n = O(1)
```

Метод бухгалтерского учёта (accounting)

Идея:
Некоторым операциям **назначается завышенная цена**, излишек сохраняется и используется для оплаты дорогих операций. filecite turn6file3

Для `push_back`:

- берём цену **3**
- 1 — за вставку
- 2 — «откладываются» на будущее копирование

Когда происходит расширение массива, накопленные «деньги» покрывают копирование.
 амортизированная сложность `O(1)`.

Метод потенциалов

Идея:
Вводится **потенциальная функция Φ**, отражающая «запас работы» структуры данных.
Амортизированная стоимость: $\hat{c} = c + \Delta\Phi$

Для динамического массива (из лекций):

```
Φ = 2·size - capacity
```

- при обычной вставке Φ увеличивается немного
- при перераспределении Φ резко уменьшается и компенсирует большую реальную стоимость

 амортизированная стоимость `push_back` равна `O(1)`.

Итог

- Амортизированная сложность ≠ средняя по вероятности
- Она гарантирует верхнюю оценку на **последовательность операций**
- Для динамического массива push_back имеет амортизированную сложность **O(1)**
- Анализ возможен методами: **агрегирования, бухгалтерского учёта, потенциалов**

11. Как реализовать очередь с помощью двух стеков? Проведите амортизированный анализ операций добавления и извлечения элемента. Что такое «кольцевой буфер» и как он решает проблему наивной реализации очереди на массиве?

Очередь с помощью двух стеков

Идея (из файла):
Очередь реализуется с использованием **двух стеков** S1 и S2.

- S1 — стек для добавления элементов
- S2 — стек для извлечения элементов

Операции

Добавление (push)

```
enqueue(x)
    S1.push(x)
```

Извлечение (pop)

```
dequeue()
    if S2 is empty
        while S1 is not empty
            S2.push(S1.pop())
    return S2.pop()
```

Амортизированный анализ

Из файла:

- Каждый элемент:
  - один раз добавляется в S1
  - один раз перекладывается из S1 в S2
  - один раз удаляется из S2
- Всего **O(1)** операций стека на элемент

Вывод:
Амортизированная сложность enqueue и dequeue равна **O(1)**,
хотя отдельная операция dequeue может стоить **O(n)**.

Кольцевой буфер

Определение:
Кольцевой буфер — это реализация очереди на массиве, где конец массива логически соединён с началом.

Используются два индекса:

- `head` — начало очереди
- `tail` — конец очереди

Как решает проблему наивной реализации

Проблема наивной очереди на массиве:
При извлечении элемента требуется **сдвиг всех элементов**, что даёт `O(n)`.

Решение кольцевого буфера:
Индексы изменяются по модулю размера массива:

```
head = (head + 1) mod n
tail = (tail + 1) mod n
```

Элементы **не сдвигаются**, все операции выполняются за **O(1)**.

Лекция 5. Система непересекающихся множеств.

12. Какие две основные операции поддерживает система непересекающихся множеств? Приведите примеры задач, где она применяется. Опишите наивную реализацию системы непересекающихся множеств и её недостатки.

Система непересекающихся множеств (DSU, Union-Find)

Две основные операции:

- `make_set(x)` — создать множество, состоящее из одного элемента x
- `union(x, y)` — объединить множества, содержащие x и y
- `find(x)` — найти представителя (лидера) множества, содержащего x

(В лекциях подчёркивается, что ключевые — `union` и `find`, `make_set` используется для инициализации.)

Примеры задач применения

Из лекций:

- проверка, находятся ли две вершины в одной компоненте связности графа
- поддержка компонент связности при добавлении рёбер
- алгоритм Краскала для поиска минимального остовного дерева

Наивная реализация

Идея:
Каждое множество хранится как **дерево**, у каждой вершины есть указатель на родителя.
Представитель множества — корень дерева.

Псевдокод

```
make_set(x)
    parent[x] ← x
```

```
find(x)
    while parent[x] ≠ x
        x ← parent[x]
    return x
```

```
union(x, y)
    rx ← find(x)
    ry ← find(y)
    if rx ≠ ry
        parent[rx] ← ry
```

Недостатки наивной реализации

Из файла:

- дерево может вырождаться в цепочку
- операция `find` в худшем случае работает за **O(n)**
- при большом числе операций общая сложность становится неэффективной

(В лекциях далее указывается, что эти недостатки устраняются эвристиками — сжатием путей и объединением по рангу, но в **наивной реализации** их нет.)

13. Что такое «ранговая эвристика» и «эвристика сжатия путей»? Как они улучшают асимптотику? Какова амортизированная временная сложность операции в системе непересекающихся множеств с использованием обеих эвристик? Что такое функция Аккермана и её обратная функция?

Ранговая эвристика (union by rank)

Определение:
При объединении двух множеств корень с **меньшим рангом** присоединяется к корню с **большим рангом**.
Ранг — оценка высоты дерева.

Псевдокод

```
union(x, y)
    rx ← find(x)
    ry ← find(y)
    if rx = ry
```

```
        return
    if rank[rx] < rank[ry]
        parent[rx] ← ry
    else if rank[rx] > rank[ry]
        parent[ry] ← rx
    else
        parent[ry] ← rx
        rank[rx] ← rank[rx] + 1
```

Эвристика сжатия путей (path compression)

Определение (из файла):
При выполнении `find(x)` все вершины на пути от x к корню **переподвешиваются напрямую к корню**.

Псевдокод

```
find(x)
    if parent[x] ≠ x
        parent[x] ← find(parent[x])
    return parent[x]
```

Как они улучшают асимптотику

Из файла:

- ранговая эвристика **не даёт деревьям вырастать высокими**;
- сжатие путей **резко уменьшает высоту** при повторных вызовах `find`;
- совместное использование делает деревья почти плоскими.

Амортизированная сложность операций DSU с обеими эвристиками

Результат (из лекций):
Амортизированная сложность операций `make_set`, `find`, `union` равна
$O(\alpha(n))$

где $\alpha(n)$ — обратная функция Аккермана.

Функция Аккермана и обратная функция

Функция Аккермана (A):
Очень быстро растущая рекурсивная функция, определяемая по уровням.

Обратная функция Аккермана ($\alpha(n)$):
Наименьшее число, для которого значение функции Аккермана превышает n.
Растёт **чрезвычайно медленно**; для всех практических n

$$\alpha(n) \leq 4$$

Лекции 6-8. Динамическое программирование.

14. Сформулируйте задачу о рюкзаке. Приведите рекуррентное соотношение и объясните его. Как можно оптимизировать решение задачи о рюкзаке по памяти? Почему внутренний цикл при этом нужно проходить в обратном порядке?

Задача о рюкзаке (0/1 knapsack)

Формулировка (из файла):
Даны n предметов, каждый с весом `w[i]` и ценностью `v[i]`, и рюкзак вместимости `W`.
Нужно выбрать подмножество предметов (каждый можно взять **не более одного раза**), чтобы **суммарный вес ≤ W**, а **суммарная ценность — максимальна**.

Рекуррентное соотношение

Обозначим `dp[i][j]` — максимальная ценность, используя **первые i предметов** и вместимость `j`.

```
dp[0][j] = 0
```

```
dp[i][j] =
    dp[i-1][j],                         если j < w[i]
    max(dp[i-1][j],
        dp[i-1][j-w[i]] + v[i]),        если j ≥ w[i]
```

Объяснение:

- либо предмет `i` не берём;
- либо берём его и добавляем его ценность к оптимальному решению для остаточной вместимости. fileciteturn10file1

Оптимизация по памяти

Из файла:
Можно хранить **одномерный массив**:

```
dp[j] — максимум для вместимости j
```

Переход:

```
for i = 1..n
    for j = W down to w[i]
        dp[j] = max(dp[j], dp[j - w[i]] + v[i])
```

Почему внутренний цикл идёт в обратном порядке

Обоснование из файла:

- если идти **вперёд**, `dp[j - w[i]]` уже может быть обновлён на текущем `i`;
- это приведёт к **многократному использованию одного и того же предмета**;
- обратный порядок гарантирует, что каждый предмет учитывается **ровно один раз**.

15. Как восстановить набор предметов, входящих в решение задачи о рюкзаке, по заполненной таблице динамического программирования? Чем решение задачи о рюкзаке с неограниченным количеством предметов отличается от классического 0/1 случая?

Восстановление набора предметов (0/1 рюкзак)

Идея (из файла):

По заполненной таблице `dp[i][j]` идём **обратно** от `dp[n][W]` и определяем, брался ли предмет `i`.

Правило восстановления:

- если `dp[i][j] = dp[i-1][j]`, то предмет `i` **не взят**;
- если `dp[i][j] = dp[i-1][j-w[i]] + v[i]`, то предмет `i` **взят**, и `j ← j - w[i]`.

Псевдокод

```
i ← n
j ← W
while i > 0 and j ≥ 0
    if dp[i][j] = dp[i-1][j]
        i ← i - 1
    else
        take item i
        j ← j - w[i]
        i ← i - 1
```

Рюкзак с неограниченным количеством предметов (unbounded knapsack)

Отличие от 0/1 случая:

- каждый предмет можно брать **неограниченное число раз**;
- в рекуррентной формуле используется **то же i**, а не `i-1`.

Рекуррентное соотношение

```
dp[i][j] =
    dp[i-1][j],                          если j < w[i]
    max(dp[i-1][j],
        dp[i][j-w[i]] + v[i]),           если j ≥ w[i]
```

Одномерная оптимизация и порядок обхода

Для неограниченного рюкзака:

```
for i = 1..n
    for j = w[i]..W
        dp[j] = max(dp[j], dp[j - w[i]] + v[i])
```

Ключевое отличие:

- внутренний цикл идёт **вперёд**,
- это позволяет использовать один и тот же предмет **много раз**. ⍰filecite⍰turn11file2⍰

Готов продолжать, если будут ещё вопросы.

16. Что такое «динамическое программирование по подмножествам»? Приведите пример задачи, например, коммивояжёра. Что такое «динамическое программирование по профилю»? Приведите пример задачи, например, замощение доминошками.

Динамическое программирование по подмножествам

Определение (из файла):
Динамическое программирование по подмножествам применяется, когда состояние определяется **множеством уже выбранных элементов**, обычно кодируемым **битовой маской**.

Состояние имеет вид:

```
dp[mask][v]
```

где `mask` — подмножество элементов, `v` — дополнительный параметр (например, текущая вершина).

Пример: задача коммивояжёра (TSP)

Формулировка (из файла):
Найти кратчайший путь, проходящий через все вершины графа ровно один раз и возвращающийся в начальную.

DP-состояние:
`dp[mask][v]` — минимальная длина пути, который:

- посещает все вершины из `mask`,
- заканчивается в вершине `v`.

Переход (идея):

```
dp[mask][v] = min over u in mask\{v} (
    dp[mask without v][u] + dist[u][v]
)
```

Сложность:
`O(n^2 · 2^n)` по времени и `O(n · 2^n)` по памяти.

Динамическое программирование по профилю

Определение (из файла):
ДП по профилю используется для задач на **решётках**, где состояние описывает **текущую строку (или столбец)** с помощью битовой маски — профиля.

Состояние:

```
dp[row][mask]
```

где `mask` кодирует занятость ячеек в текущей строке.

Пример: замощение прямоугольника доминошками

Задача (из файла):
Посчитать число способов замостить прямоугольник доминошками 2×1. filecite turn12file3

Идея ДП по профилю:

- перебираем строки по очереди;

- `mask` показывает, какие клетки текущей строки уже заняты;
- переходы осуществляются расстановкой доминошек и формированием профиля следующей строки.

Ключевая разница

- **По подмножествам** — состояния это *наборы элементов* (`mask` = выбранные объекты)
- **По профилю** — состояния это *локальные конфигурации строки/столбца* (`mask` = профиль решётки)

Лекция 9. Дерево отрезков.

17. Что такое «дерево отрезков»? Какие операции и с какой сложностью оно поддерживает? Опишите структуру дерева отрезков и принцип его хранения в массиве.

Дерево отрезков

Определение (из файла):
Дерево отрезков — это структура данных для хранения массива, позволяющая **быстро отвечать на запросы на отрезках** и выполнять **изменения элементов**. Каждый узел соответствует некоторому подотрезку массива.

Поддерживаемые операции и их сложность

- запрос на отрезке (например, `min / max / sum` на `[l, r]`) — **O(log n)**
- изменение одного элемента — **O(log n)**
- построение дерева — **O(n)**

Структура дерева отрезков

Идея:

- корень соответствует отрезку `[0, n-1]`;
- каждый внутренний узел делит свой отрезок на два: `[l, mid]` и `[mid+1, r]`;
- листья соответствуют одиночным элементам массива.

Значение в узле — результат операции (сумма, минимум и т.п.) на его отрезке.

Хранение дерева отрезков в массиве

Из файла:

- дерево хранится в массиве `t`;

- для вершины с индексом `v`:

  - левый сын: $2 \cdot v$
  - правый сын: $2 \cdot v + 1$

- корень хранится в `t[1]`;
- размер массива — порядка $4 \cdot n$, что достаточно для хранения всего дерева.

18. Как в дереве отрезков реализованы операции запроса на отрезке и обновления элемента? В чём заключается идея «отложенных обновлений» в дереве отрезков? В каких случаях она необходима?

Запрос на отрезке и обновление элемента в дереве отрезков

Запрос на отрезке (из файла):
Рекурсивно спускаемся по дереву. Для узла с отрезком `[l, r]` и запроса `[ql, qr]`:

- если отрезки **не пересекаются** — возвращаем нейтральный элемент;
- если `[l, r]` **целиком внутри** `[ql, qr]` — возвращаем значение узла;
- иначе — делим запрос на детей и объединяем ответы.

Псевдокод

```
query(v, l, r, ql, qr)
    if r < ql or qr < l
        return NEUTRAL
    if ql ≤ l and r ≤ qr
        return t[v]
    mid ← (l + r) div 2
    return combine(
        query(2v, l, mid, ql, qr),
        query(2v+1, mid+1, r, ql, qr)
    )
```

Сложность: $O(\log n)$ — посещается ограниченное число вершин.

Обновление одного элемента (из файла):
Спускаемся к листу, соответствующему позиции `pos`, меняем значение и пересчитываем путь вверх.

Псевдокод

```
update(v, l, r, pos, new_val)
    if l = r
        t[v] ← new_val
        return
    mid ← (l + r) div 2
    if pos ≤ mid
        update(2v, l, mid, pos, new_val)
    else
        update(2v+1, mid+1, r, pos, new_val)
    t[v] ← combine(t[2v], t[2v+1])
```

Сложность: $O(\log n)$.

Отложенные обновления (lazy propagation)

Идея (из файла):
При обновлении **целого отрезка** изменение **не проталкивается сразу** к листьям.
В узле сохраняется «отложенное» обновление, которое будет применено к детям **только при необходимости** (перед следующим спуском).

Используется дополнительный массив `lazy`.

Принцип:

- если обновляемый отрезок **полностью покрывает** узел — обновляем `t[v]` и записываем пометку в `lazy[v]`;
- при дальнейшем спуске пометка «проталкивается» к детям.

Когда это необходимо

Из файла:

- при **массовых обновлениях на отрезках** (прибавить, присвоить и т.п.);
- без отложенных обновлений каждое такое обновление стоило бы `O(n)`;
- с lazy propagation и запросы, и обновления работают за **O(log n)**.

Лекция 10. Дерево Фенвика. Разреженная таблица.

19. Что такое «дерево Фенвика»? Какие операции оно поддерживает и в чём его преимущества перед деревом отрезков? Объясните, как работает операция вычисления суммы на префиксе в дереве Фенвика, используя операцию & i i− .

Дерево Фенвика (Binary Indexed Tree)

Определение (из файла):
Дерево Фенвика — это структура данных для работы с массивом, позволяющая эффективно считать **префиксные суммы** и выполнять **точечные обновления**.

Поддерживаемые операции и их сложность

Из файла:

- `sum(i)` — сумма элементов на префиксе `[1..i]` — **O(log n)**
- `add(i, x)` — прибавить x к элементу с индексом `i` — **O(log n)**
- построение дерева — **O(n log n)** (или O(n)` специальным способом)

Преимущества перед деревом отрезков

Из файла:

- проще в реализации;
- требует **O(n)** памяти (против ~4n у дерева отрезков);
- меньшая константа во времени;
- эффективно работает именно для **префиксных сумм**.

(Отмечено, что дерево Фенвика поддерживает не все операции, в отличие от дерева отрезков.)

Структура и идея хранения

Из файла:
Используется массив `t[1..n]`, где
`t[i]` хранит сумму элементов на отрезке

```
(i - (i & -i) + 1) .. i
```

Операция вычисления суммы на префиксе и роль `i & -i`

Идея (из файла):
Операция `i & -i` выделяет **младший установленный бит** числа `i`.
Он определяет длину отрезка, за который отвечает `t[i]`.

Псевдокод вычисления префиксной суммы

```
sum(i)
    res ← 0
    while i > 0
        res ← res + t[i]
        i ← i - (i & -i)
    return res
```

Объяснение:
На каждом шаге мы переходим к следующему узлу, который хранит сумму для **предыдущего блока** элементов, постепенно покрывая весь префикс `[1..i]`.

20. Для каких задач предназначена разреженная таблица? Каковы её ограничения? Опишите процесс предобработки и выполнения запроса в разреженной таблице для задачи поиска минимума на отрезке. Почему операция запроса выполняется за O(1) неприменима для обновлений?

Разреженная таблица (Sparse Table)

Назначение (из файла):
Разреженная таблица применяется для задач **поиска минимума на отрезке (RMQ)** в **статическом массиве**, то есть **без обновлений**. filecite turn16file0

Ограничения

Из файла:

- массив **не должен изменяться**;
- структура **не поддерживает обновления**;
- подходит только для идемпотентных операций (например, `min`, `max`).

Предобработка

Идея (из файла):
Строится таблица:

```
st[i][k] — минимум на отрезке длины 2^k,
           начинающемся в позиции i
```

База

```
st[i][0] = a[i]
```

Переход

```
st[i][k] = min(st[i][k-1],
               st[i + 2^(k-1)][k-1])
```

Сложность предобработки:
`O(n log n)` по времени и памяти.

Запрос минимума на отрезке

Для запроса `[l, r]`:

- `k = floor(log2(r - l + 1))`
- ответ:

```
min(st[l][k],
    st[r - 2^k + 1][k])
```

Сложность запроса: O(1).

Почему запрос O(1), но нет обновлений

Из лекции:

- минимум на отрезке можно получить из **двух заранее посчитанных блоков**;
- при изменении одного элемента нужно пересчитать **множество значений таблицы**, что требует `O(n log n)`;
- поэтому разреженная таблица **неприменима для обновлений**.

Лекция 11. Сбалансированные деревья поиска.

21. Что такое «сбалансированное дерево поиска» и зачем оно нужно? В чём проблема несбалансированного двоичного дерева поиска?

Сбалансированное дерево поиска

Определение (из файла):
Сбалансированное дерево поиска — это двоичное дерево поиска, у которого **высота поддерживается логарифмической** за счёт специальных правил балансировки.

Зачем нужно:
Оно обеспечивает выполнение основных операций поиска, вставки и удаления за **O(log n)**.

Проблема несбалансированного двоичного дерева поиска

Суть проблемы:
В обычном двоичном дереве поиска при неудачном порядке вставки дерево может **выродиться в цепочку**.

В этом случае:

- высота дерева становится **O(n)**;
- операции `search`, `insert`, `delete` работают за **O(n)** вместо `O(log n)`.

Вывод

- несбалансированное ДДП не гарантирует логарифмическую сложность;
- сбалансированные деревья устраняют вырождение и гарантируют `O(log n)` для операций.

22. Опишите инвариант АВЛ-дерева. Какие повороты используются для поддержания его сбалансированности? Опишите основную идею Сплей-дерева. Что такое операция подъёма к корню и какие шаги она включает?

АВЛ-дерево: инвариант и повороты

Инвариант АВЛ-дерева (из файла):
Для любой вершины высоты её левого и правого поддеревьев отличаются **не более чем на 1**:
$|h(left) - h(right)| \leq 1$

Это гарантирует, что высота дерева равна **O(log n)**.

Повороты в АВЛ-дереве

Используемые повороты (из файла):

1. **Малый правый поворот (LL)**
2. **Малый левый поворот (RR)**
3. **Большой левый-правый (LR)**
4. **Большой правый-левый (RL)**

Выбор поворота зависит от того, в каком поддереве произошло нарушение баланса.

Сплей-дерево: основная идея

Определение (из файла):
Сплей-дерево — это двоичное дерево поиска, которое **самоадаптируется**:
каждый раз при обращении к вершине она **поднимается в корень**.

Баланс **явно не хранится**, но за счёт перестроений достигается хорошая амортизированная асимптотика.

Операция подъёма к корню (splay)

Суть (из файла):
Операция `splay(x)` поднимает вершину `x` в корень дерева серией поворотов.

Основные шаги (случаи):

1. **Zig**
   x — ребёнок корня → один поворот

2. **Zig-Zig**
   x и его родитель — оба левые или оба правые дети → два однотипных поворота

3. **Zig-Zag**
   x — левый ребёнок, родитель — правый (или наоборот) → два разных поворота

23. Сравните АВЛ-дерево и Сплей-дерево. В чём преимущества и недостатки каждого?

Сравнение АВЛ-дерева и Сплей-дерева

АВЛ-дерево

Характеристика (из файла):
АВЛ-дерево — строго сбалансированное двоичное дерево поиска с инвариантом $|h_L - h_R| \leq 1$.

Преимущества:

- гарантированная высота **O(log n)**;
- операции `search`, `insert`, `delete` — **O(log n)** в худшем случае;
- предсказуемое время работы.

Недостатки:

- сложная реализация;
- необходимость хранить и обновлять высоты/балансы;
- больше поворотов при вставках и удалениях.

Сплей-дерево

Характеристика (из файла):
Сплей-дерево — самоадаптирующееся двоичное дерево поиска, где при каждом доступе элемент поднимается в корень (операция `splay`). Баланс явно не поддерживается.

Преимущества:

- простая структура (нет хранения высот/рангов);
- хорошая **амортизированная** сложность операций;
- часто используемые элементы оказываются ближе к корню.

Недостатки:

- нет гарантии **O(log n)** для одной операции;
- возможны медленные отдельные операции;
- форма дерева может сильно меняться.

Асимптотика (из файла)

- **АВЛ-дерево:** `O(log n)` **в худшем случае** для операций.
- **Сплей-дерево:** `O(log n)` **амортизированно**, но не в худшем случае.

Лекция 12. Декартово дерево по неявному ключу.

24. Что такое декартово дерево? Какие два свойства оно сочетает?

Декартово дерево (treap)

Определение (из файла):
Декартово дерево — это структура данных, которая является **двоичным деревом поиска по ключам** и одновременно **кучей по приоритетам**.

Какие два свойства оно сочетает

Из лекциям:

1. **Свойство двоичного дерева поиска (BST)** — по ключу x:

- все ключи в левом поддереве меньше x;
- все ключи в правом поддереве больше x.

2. **Свойство кучи (heap)** — по приоритету y:

- приоритет в вершине **не больше** (или не меньше — в зависимости от варианта) приоритетов в её детях
  (обычно — min-heap по y).

Небольшая реализация **декартова дерева по неявному ключу (Treap)** в псевдокоде (ключевые операции `merge`, `split`, `insert`, `delete`) — как в файле.

text

```
# Вспомогательные
size(node):
    return node.size if node != null else 0

update_size(node):
    node.size = 1 + size(node.left) + size(node.right)

# merge(A, B): все элементы A должны быть "левее" всех элементов B
merge(A, B):
    if A == null: return B
    if B == null: return A
    if A.priority > B.priority:
        A.right = merge(A.right, B)
        update_size(A)
        return A
    else:
        B.left = merge(A, B.left)
        update_size(B)
        return B

# split(T, k): L содержит первые k элементов, R — остальные
```

```
split(T, k):
    if T == null: return (null, null)

    left_size = size(T.left)

    if k <= left_size:
        (L, R) = split(T.left, k)
        T.left = R
        update_size(T)
        return (L, T)
    else:
        (L, R) = split(T.right, k - left_size - 1)
        T.right = L
        update_size(T)
        return (T, R)

# Вставка по позиции pos (0-index)
insert(T, pos, value, priority):
    (L, R) = split(T, pos)
    new_node = Node(value, priority)
    return merge(merge(L, new_node), R)

# Удаление элемента на позиции pos
delete(T, pos):
    (L, R) = split(T, pos)
    (M, R2) = split(R, 1)    # M — удаляемый узел
    return merge(L, R2)
```

25. Объясните операции слияния и разделения в декартовом дереве. Какова их временная сложность? Как декартово дерево с неявным ключом может использоваться для работы с последовательностями? Приведите примеры операций.

Операции **слияния** и **разделения** в декартовом дереве

Слияние (merge)

Идея (из файла):

`merge(A, B)` объединяет **два декартовых дерева**, при этом предполагается, что **все ключи A меньше всех ключей B**.

Корнем результата становится вершина с **большим приоритетом**, а слияние рекурсивно продолжается в соответствующем поддереве.

Псевдокод (как в файле):

```
merge(A, B)
    if A = null return B
    if B = null return A
    if A.priority > B.priority
        A.right = merge(A.right, B)
        return A
    else
        B.left = merge(A, B.left)
        return B
```

Разделение (split)

Идея (из файла):
`split(T, x)` делит дерево T на два:

- L — все элементы с ключами < x
- R — все элементы с ключами ≥ x

Разделение идёт рекурсивно, сравнивая x с ключом текущей вершины.

Псевдокод:

```
split(T, x)
    if T = null
        return (null, null)
    if key(T) < x
        (L, R) = split(T.right, x)
        T.right = L
        return (T, R)
    else
        (L, R) = split(T.left, x)
        T.left = R
        return (L, T)
```

Временная сложность

Из файла:

- `merge` — **O(h)**
- `split` — **O(h)**

где h — высота декартова дерева.
Так как приоритеты выбираются случайно, **математическое ожидание высоты** равно `O(log n)`, значит:

```
merge, split – O(log n) в среднем
```

Декартово дерево с неявным ключом

Идея (из файла):
Ключ **явно не хранится**.
Позиция элемента в последовательности определяется:

```
key = size(left_subtree)
```

То есть **порядком обхода in-order**.

В вершине дополнительно хранится `size` поддерева.

Использование для работы с последовательностями

Из файла:

С неявным ключом декартово дерево позволяет работать с последовательностью как с массивом, поддерживая операции за `O(log n)`:

Примеры операций:

- вставка элемента в позицию `i`;
- удаление элемента по позиции;
- получение элемента по индексу;
- вырезание подотрезка `[l, r]`;
- вставка подотрезка в другое место;
- разворот подотрезка (с доп. флагом).

Все они реализуются через комбинации `split` и `merge`.

26. Сравните декартово дерево с другими структурами данных, например, с деревом отрезков, с точки зрения операций и сложности.

Сравнение декартова дерева и дерева отрезков

Декартово дерево (Treap)

Характеристика (из файла):
Декартово дерево сочетает **BST по ключу** и **кучу по приоритету**; при случайных приоритетах имеет **ожидаемую высоту O(log n)**.

Поддерживаемые операции и сложность (в среднем):

- поиск / вставка / удаление — **O(log n)**;
- `split`, `merge` — **O(log n)**;
- с неявным ключом: операции над последовательностями (вставка/удаление по позиции, вырезание отрезка, склейка) — **O(log n)**.

Плюсы:

- гибкость (удобно работать с последовательностями);
- поддержка сложных операций через `split/merge`;
- простая модификация под новые операции.

Минусы:

- нет гарантии логарифмической сложности в худшем случае (только в среднем);
- зависит от случайных приоритетов.

Дерево отрезков

Характеристика (из файла):
Дерево отрезков — структура для запросов и обновлений **на отрезках массива**.

Поддерживаемые операции и сложность:

- запрос на отрезке — **O(log n)**;
- точечное обновление — **O(log n)**;
- отложенные обновления (range update) — **O(log n)**;
- построение — **O(n)**.

Плюсы:

- гарантированная сложность **O(log n)** в худшем случае;
- хорошо подходит для диапазонных запросов (sum/min/max и т.п.).

Минусы:

- менее гибко для операций со структурой массива (вставки/удаления элементов);
- больший расход памяти (~4n).

Сравнение по назначению (из файла)

- **Декартово дерево** — лучше для **динамических последовательностей**, когда важны вставки, удаления и перестановки элементов. filecite turn22file1
- **Дерево отрезков** — лучше для **частых запросов на отрезках** при фиксированном размере массива. filecite turn22file4

Лекция 13. Наименьший общий предок.

27. Дайте определение наименьшего общего предка двух узлов в дереве. Приведите пример. Опишите метод бинарных подъёмов. Какая предобработка требуется и какова сложность запроса?

Наименьший общий предок (LCA)

Определение (из файла):
Наименьший общий предок двух вершин u и v в дереве — это **вершина наибольшей глубины**, которая является предком **и u, и v**.

Пример:
Если вершина x — предок u и v, и никакой её потомок уже не является общим предком, то x — LCA(u, v).

Метод бинарных подъёмов

Идея (из файла):
Для каждой вершины заранее хранятся её предки на расстояниях
$2^0, 2^1, 2^2, \ldots$ Это позволяет «поднимать» вершину вверх по дереву **скачками степеней двойки**.

Предобработка

Из файла:

1. Выполняется DFS/BFS от корня.
2. Для каждой вершины v сохраняется:
   - `up[v][0]` — её непосредственный родитель;
   - `up[v][k]` — предок на расстоянии 2^k.

Рекуррентно:

```
up[v][k] = up[ up[v][k-1] ][k-1]
```

Также сохраняется глубина `depth[v]`.

Сложность предобработки:

$$O(n\log n)$$

Запрос LCA(u, v)

Из файла:

1. Если `depth[u] < depth[v]`, поменять их местами.
2. Поднять u до уровня v, используя бинарные подъёмы.
3. Если u = v, это и есть LCA.
4. Иначе одновременно поднимать u и v от больших k к меньшим, пока их предки различны.
5. Ответ — `parent[u]`.

Сложность запроса

Результат (из файла):

- время одного запроса LCA — **O(log n)**;
- память — **O(n log n)**.

28. Как задача нахождения НОП сводится к поиску минимума на отрезке? Опишите алгоритм, использующий эйлеров обход и разреженную таблицу. Объясните алгоритм Фарах-Колтона и Бендера и сравните методы нахождения НОП по времени и памяти.

Сведение задачи НОП (LCA) к поиску минимума на отрезке (RMQ)

Идея (из файла):
Задача нахождения наименьшего общего предка двух вершин в дереве сводится к задаче **поиска минимума на отрезке** по массиву глубин, полученному из **эйлерова обхода дерева**.

Эйлеров обход

Из файла:

Выполняется DFS, при котором:

- при **каждом посещении вершины** она добавляется в массив `euler`;
- параллельно сохраняется массив `depth` — глубина каждой записи в `euler`;
- для каждой вершины v запоминается `first[v]` — **позиция первого появления** v в `euler`.

Длина массива `euler` равна `2n - 1`.

Как получается НОП через RMQ

Из файла:

Для двух вершин u и v:

1. берём их первые вхождения `l = first[u]`, `r = first[v]`;
2. рассматриваем отрезок `euler[l..r]`;
3. **вершина с минимальной глубиной** на этом отрезке и есть `LCA(u, v)`.

Таким образом:

```
LCA(u, v) = argmin depth[i] for i in [first[u], first[v]]
```

Алгоритм с разреженной таблицей

Из файла:

1. Выполнить эйлеров обход — `O(n)`
2. Построить разреженную таблицу по массиву `depth` — `O(n log n)`
3. Каждый запрос RMQ (и, значит, LCA) — **O(1)**

Итог:

- предобработка: `O(n log n)`
- запрос LCA: `O(1)`
- память: `O(n log n)`

Алгоритм Фарах–Колтона и Бендера

Идея (из файла):
Это улучшение метода RMQ для массива глубин из эйлерова обхода, использующее то, что **разности соседних элементов равны ±1**. ⍰filecite⍰turn24file4⍰

Массив разбивается на **блоки** фиксированного размера, и:

- внутри блоков RMQ решается за `O(1)` по предвычисленным таблицам;
- между блоками — также через RMQ по их минимумам. ⍰filecite⍰turn24file4⍰

Сложности алгоритма Фарах–Колтона и Бендера

Из файла:

- предобработка: **O(n)**
- запрос LCA: **O(1)**
- память: **O(n)**

Сравнение методов (из файла)

Метод	Предобработка	Запрос	Память
Бинарные подъёмы	`O(n log n)`	`O(log n)`	`O(n log n)`
Эйлеров обход + Sparse Table	`O(n log n)`	`O(1)`	`O(n log n)`
Фарах–Колтон–Бендер	`O(n)`	`O(1)`	`O(n)`

Лекция 14. Heavy-Light декомпозиция. Центроидная декомпозиция.

29. Что такое Heavy-Light декомпозиция? Каковы её ключевые понятия? Опишите шаги построения Heavy-Light декомпозиции. Какие свойства дерева используются?

Heavy-Light декомпозиция (HLD)

Определение (из файла):
Heavy-Light декомпозиция — это способ разбиения дерева на **пути**, позволяющий свести запросы на пути между вершинами к **логарифмическому числу запросов на отрезках**.

Ключевые понятия

Из файла:

- **Размер поддерева** `sz[v]` — число вершин в поддереве вершины `v`.
- **Тяжёлое ребро (heavy edge)** — ребро `(v, u)`, где `u` — сын `v` с **максимальным поддеревом**.
- **Лёгкое ребро (light edge)** — любое другое ребро.
- **Тяжёлый путь** — последовательность вершин, соединённых тяжёлыми рёбрами.

Каждая вершина принадлежит **ровно одному** тяжёлому пути.

Шаги построения Heavy-Light декомпозиции

Из файла:

1. **DFS от корня**
   - вычисляются `sz[v]` — размеры поддеревьев;
   - для каждой вершины выбирается её тяжёлый сын.
2. **Разбиение на пути**
   - тяжёлые рёбра образуют тяжёлые пути;
   - при переходе по лёгкому ребру начинается **новый путь**.
3. **Нумерация вершин**
   - вершины каждого тяжёлого пути нумеруются **подряд**;
   - каждому пути сопоставляется отрезок в массиве.

После этого запросы на пути в дереве сводятся к запросам на отрезках массива.

Используемые свойства дерева

Из файла:

- у каждой вершины **не более одного** тяжёлого сына;
- при переходе по лёгкому ребру размер поддерева уменьшается **как минимум в 2 раза**;
- на любом пути от вершины к корню встречается не более **O(log n)** лёгких рёбер.

Отсюда следует, что любой путь в дереве разбивается на **O(log n)** тяжёлых путей.

30. Как HLD используется для выполнения запросов на путях в дереве? Приведите пример.

Использование Heavy-Light декомпозиции (HLD) для запросов на путях

Идея (из файла):
HLD позволяет разбить путь между двумя вершинами в дереве на **O(log n)** непересекающихся отрезков, каждый из которых соответствует **непрерывному отрезку** в линейном массиве. Тогда запрос на пути сводится к нескольким запросам на отрезках.

Как выполняется запрос на пути `u – v`

Из файла:

1. Пока вершины u и v находятся **в разных тяжёлых путях**:
   - выбираем вершину с **большей глубиной головы пути**;
   - добавляем запрос на отрезке от этой вершины до головы её пути;
   - поднимаем вершину к родителю головы пути (переход по лёгкому ребру).
2. Когда u и v оказываются **в одном тяжёлом пути**:

– выполняем **один запрос** на отрезке между их позициями в массиве.

Все запросы объединяются (суммируются, берётся минимум и т.п.).

Пример (из файла)

Задача:
Найти сумму значений вершин на пути между u и v.

Решение с HLD:

- после HLD каждой вершине сопоставлен индекс в массиве;
- над массивом построено **дерево отрезков**;
- путь u – v разбивается на O(log n) отрезков массива;
- для каждого отрезка выполняется запрос суммы в дереве отрезков.

Сложность

- число отрезков на пути — **O(log n)**;
- каждый запрос на отрезке — **O(log n)**;

Итоговая сложность запроса на пути:
$O(\log^2 n)$ (или O(log n) при использовании структур с O(1) RMQ).

31. Что такое центроидная декомпозиция? Как находится центроид? Опишите алгоритм построения центроидной декомпозиции и сравните её с HLD.

Центроидная декомпозиция

Определение (из файла):
Центроидная декомпозиция — это рекурсивное разбиение дерева по **центроидам**, при котором строится новое дерево (дерево центроидов), отражающее иерархию разбиений.

Что такое центроид и как он находится

Центроид дерева (из файла):
Центроид — это вершина, удаление которой разбивает дерево на компоненты, каждая из которых имеет размер **не более n/2**, где n — размер текущего дерева.

Как найти центроид:

1. выполнить DFS и посчитать размеры поддеревьев;
2. для каждой вершины проверить, что размер **каждого** поддерева ≤ n/2;
3. такая вершина и есть центроид (он существует и может быть не единственным).

Алгоритм построения центроидной декомпозиции

Из файла:

1. Найти центроид текущего дерева.
2. Сделать его корнем центроидного дерева.

3. Удалить центроид, получив несколько компонент.

4. Рекурсивно применить алгоритм к каждой компоненте.

Рекурсия продолжается, пока компонента не станет пустой.

Глубина центроидного дерева: `O(log n)`.

Сравнение центроидной декомпозиции и HLD

Центроидная декомпозиция:

- ориентирована на **задачи со всеми расстояниями до поддеревьев** или глобальные запросы;
- дерево делится **по вершинам**, а не по путям;
- глубина разбиения — `O(log n)`;
- часто используется для задач вида «для каждой вершины найти что-то относительно всех остальных».

Heavy-Light декомпозиция (HLD):

- ориентирована на **запросы на путях**;
- разбивает дерево на **тяжёлые пути**;
- путь между вершинами делится на `O(log n)` отрезков;
- хорошо сочетается с деревом отрезков.

Ключевое различие (из файла)

- **HLD** — для запросов **на путях** (u–v).
- **Центроидная декомпозиция** — для запросов, зависящих от **расстояний и поддеревьев относительно центра**.

Лекции 15-17. DFS, компоненты, мосты, Эйлеров цикл.

32. Опишите алгоритм обхода в глубину для графа. Какова его сложность? Как с помощью обхода в глубину найти цикл в неориентированном и ориентированном графах?

Обход в глубину (DFS)

Определение (из файла):
Обход в глубину — это алгоритм обхода графа, при котором из текущей вершины рекурсивно посещается **любой ещё не посещённый сосед**, пока это возможно, после чего происходит возврат назад.

Алгоритм DFS

Псевдокод

```
dfs(v)
    used[v] ← true
    for each u in adj[v]
        if not used[u]
            dfs(u)
```

Обход запускается для всех вершин (если граф несвязный).

Сложность DFS

Из файла:

- по времени: **O(n + m)**
  где n — число вершин, m — число рёбер;
- по памяти: **O(n)** (массив посещённых вершин + стек рекурсии).

Поиск цикла в **неориентированном** графе

Идея (из файла):
Во время DFS для каждой вершины запоминается её **родитель**.
Если из вершины v есть ребро в уже посещённую вершину u,
и u **не является родителем v**, то найден цикл.

Условие цикла:

```
used[u] = true and u ≠ parent[v]
```

Поиск цикла в **ориентированном** графе

Идея (из файла):
Используется раскраска вершин:

- 0 — не посещена
- 1 — в текущем стеке рекурсии
- 2 — обработана полностью

Псевдокод

```
dfs(v)
    color[v] ← 1
    for each u in adj[v]
        if color[u] = 0
            dfs(u)
        else if color[u] = 1
            cycle found
    color[v] ← 2
```

Пояснение:
Ребро в вершину с цветом 1 — это **обратное ребро**, оно образует цикл.

Кратко:

- DFS обходит граф, углубляясь максимально глубоко
- сложность: **O(n + m)**
- цикл в неориентированном графе — ребро к посещённой вершине, не являющейся родителем
- цикл в ориентированном графе — обратное ребро в вершину из стека рекурсии

33. Что такое топологическая сортировка? Приведите алгоритм Кана и алгоритм на основе обхода в глубину. Примеры применения.

Топологическая сортировка

Определение (из файла):
Топологическая сортировка — это упорядочивание вершин **ориентированного ациклического графа (DAG)** такое, что для каждого ребра u → v вершина u идёт **раньше** v.

Алгоритм Кана (по входящим степеням)

Идея (из файла):
Вершины с нулевой входящей степенью можно ставить первыми; после их удаления степени пересчитываются.

Псевдокод

```
kahn_toposort(G)
    for v in V
        indeg[v] ← number of incoming edges
    Q ← all v with indeg[v] = 0
    order ← empty list
    while Q not empty
        v ← pop(Q)
        add v to order
        for each (v → u)
            indeg[u] ← indeg[u] – 1
            if indeg[u] = 0
                push u to Q
    if |order| < |V|
        cycle exists
    return order
```

Сложность: `O(n + m)`.

Алгоритм на основе DFS

Идея (из файла):
Вершина добавляется в ответ **после обработки всех её потомков**; итоговый порядок — обратный времени выхода.

Псевдокод

```
dfs(v)
    used[v] ← true
    for each (v → u)
        if not used[u]
            dfs(u)
    add v to order

toposort_dfs(G)
    for v in V
        if not used[v]
            dfs(v)
    reverse(order)
```

Сложность: `O(n + m)`.

Примеры применения

- планирование задач с зависимостями;
- порядок компиляции модулей;
- анализ зависимостей пакетов;
- упорядочивание шагов вычислений в DAG.

Важно: топологическая сортировка существует **только для DAG**; наличие цикла означает, что сортировка невозможна.

34. Дайте определение компоненты сильной связности. Опишите алгоритм Косарайю. Что такое конденсация графа?

Компонента сильной связности (КСС)

Определение (из файла):
Компонента сильной связности ориентированного графа — это **максимальное множество вершин**, такое что для любых двух вершин u и v из этого множества существуют пути **u → v и v → u**.

Алгоритм Косарайю

Идея (из файла):
Алгоритм использует **два обхода в глубину** и обращение рёбер графа.

Шаги алгоритма:

1. Выполнить DFS в исходном графе и записать вершины **в порядке убывания времени выхода** (в стек или список).
2. Построить **транспонированный граф** (все рёбра обратить).
3. Запускать DFS в транспонированном графе, извлекая вершины в порядке из шага 1.
   Все вершины, посещённые за один DFS, образуют **одну компоненту сильной связности**.

Сложность:
Алгоритм выполняет два DFS **O(n + m)**.

Конденсация графа

Определение (из файла):
Конденсация графа — это граф, полученный из исходного **сжатием каждой компоненты сильной связности в одну вершину**.

Свойства:

- конденсация — **ориентированный ациклический граф (DAG)**;
- между двумя вершинами конденсации есть ребро, если в исходном графе было ребро между соответствующими КСС.

35. Дайте определение моста и точки сочленения. Опишите алгоритм их поиска. Что такое Эйлеров цикл? Каковы условия его существования? Опишите алгоритм нахождения Эйлерова цикла.

Мост и точка сочленения

Определения (из файла):

- **Мост** — ребро неориентированного графа, удаление которого **увеличивает число компонент связности**.
- **Точка сочленения** — вершина, удаление которой (вместе с инцидентными рёбрами) **увеличивает число компонент связности**.

Алгоритм поиска мостов и точек сочленения (DFS)

Идея (из файла):
Используется обход в глубину с временами входа `tin[v]` и значениями `low[v]` — минимальное `tin`, достижимое из v по рёбрам дерева DFS и одному обратному ребру.

Правила:

- Ребро `(v, u)` — **мост**, если

```
low[u] > tin[v]
```

- Вершина v — **точка сочленения**, если:
  - v не корень DFS и существует ребёнок u с

    ```
    low[u] ≥ tin[v]
    ```

  - v — корень DFS и имеет **более одного** ребёнка в дереве DFS. ⍰filecite⍰turn31file1⍰

Сложность: `O(n + m)`.

Эйлеров цикл

Определение (из файла):
Эйлеров цикл — это цикл в графе, проходящий **по каждому ребру ровно один раз** и возвращающийся в исходную вершину. ⍰filecite⍰turn31file2⍰

Условия существования Эйлерова цикла

Для неориентированного графа (из файла):

- граф связен (если игнорировать изолированные вершины);
- степени **всех вершин чётные**.

Для ориентированного графа (из файла):

- для каждой вершины `in-degree = out-degree`;
- все вершины с ненулевой степенью лежат в одной компоненте сильной связности.

Алгоритм нахождения Эйлерова цикла

Алгоритм Хиерхольцера (из файла):

Идея:
Пока есть неиспользованные рёбра, идём по ним, удаляя из графа, формируя цикл; при встрече вершины с неиспользованными рёбрами «вклеиваем» новый цикл.

Псевдокод (схема):

```
stack ← [start]
path ← empty
while stack not empty
    v ← top(stack)
    if v has unused edges
        take one (v,u), mark used
        push u to stack
    else
        pop v from stack
        add v to path
```

Полученный `path` (в обратном порядке) — эйлеров цикл.

Сложность: `O(n + m)`.

Лекции 18-19. Остовные деревья и кратчайшие пути.

36. Дайте определение минимального остовного дерева. Опишите алгоритмы Крускала и Прима и сравните их.

Минимальное остовное дерево (МОД, MST)

Определение (из файла):
Минимальное остовное дерево связного неориентированного взвешенного графа — это остовное дерево, **суммарный вес рёбер которого минимален**.

Алгоритм **Крускала**

Идея (из файла):
Рёбра сортируются по возрастанию веса и добавляются по одному, если они **не образуют цикл**. Для проверки используется **DSU**.

DSU (Disjoint Set Union) — структура данных для работы с **непересекающимися множествами**, поддерживает операции:

- `find` — найти представителя множества
- `union` — объединить множества

Используется в алгоритме **Крускала**.

Шаги:

1. отсортировать рёбра по весу;
2. идти по рёбрам, добавляя ребро, если его концы в разных компонентах;
3. объединять компоненты.

Сложность:
`O(m log m)` (сортировка рёбер), DSU — почти `O(1)` амортизированно.

Алгоритм **Прима**

Идея (из файла):
Строит МОД, **расширяя текущее дерево**, каждый раз добавляя **минимальное ребро**, соединяющее дерево с новой вершиной.

Шаги:

1. выбрать стартовую вершину;
2. поддерживать для каждой вершины минимальное ребро к текущему дереву;
3. на каждом шаге добавлять вершину с минимальным таким ребром.

Сложность:

- с приоритетной очередью: `O(m log n)`;
- с матрицей смежности: `O(n^2)`.

Сравнение Крускала и Прима

Из файла:

- **Крускал**:
  - работает с **рёбрами**;
  - удобен для **разреженных графов**;
  - требует DSU и сортировки рёбер.
- **Прим**:
  - работает с **вершинами**;
  - удобен для **плотных графов**;
  - не требует DSU, использует очередь с приоритетами.

Оба алгоритма корректно находят минимальное остовное дерево.

37. Перечислите основные алгоритмы поиска кратчайших путей и условия их применения. Опишите алгоритмы BFS, Дейкстры, Форда-Беллмана, Флойда-Уоршелла и Джонсона.

НИЖЕ НА ВСЯКИЙ СЛУЧАЙ ПРИМЕРЫ ВСЕХ АЛГОРИТМОВ НА ПСЕВДОКОДЕ. <u>СДЕЛАНЫ НЕ ПО ЛЕКЦИЯМ</u>

(находятся в конце ответов, нужно их писать или нет – твое дело)

Основные алгоритмы поиска кратчайших путей и условия применения

BFS (поиск в ширину)

Условие применения (из файла):
Граф **невзвешенный** или все рёбра имеют вес **1**.

Идея:
Обход графа слоями, расстояние — количество рёбер от источника.

Результат:
Кратчайшие пути от одной вершины до всех остальных.

Сложность:

- время: **O(n + m)**
- память: **O(n)**

Алгоритм Дейкстры

Условие применения (из файла):
Все веса рёбер **неотрицательные**.

Идея:
Постепенно выбирается вершина с минимальным текущим расстоянием и «фиксируется».

Сложность:

- с приоритетной очередью: **O(m log n)**

- с массивом: **$O(n^2)$**

Назначение:
Кратчайшие пути от **одного источника**.

Алгоритм Форда–Беллмана

Условие применения (из файла):
Допускаются **отрицательные веса рёбер**.

Идея:
Последовательно релаксируются все рёбра $n-1$ раз.

Особенность:
Позволяет **обнаружить отрицательный цикл**.

Сложность:

- время: **$O(n \cdot m)$**

Алгоритм Флойда–Уоршелла

Условие применения (из файла):
Граф небольшой, требуется найти кратчайшие пути **между всеми парами вершин**.

Идея:
Динамическое программирование по промежуточным вершинам.

Сложность:

- время: **$O(n^3)$**
- память: **$O(n^2)$**

Алгоритм Джонсона

Условие применения (из файла):
Взвешенный граф, возможны **отрицательные рёбра**, но **нет отрицательных циклов**.

Идея:

1. Добавляется фиктивная вершина
2. Запускается Форда–Беллман
3. Перевзвешиваются рёбра
4. Из каждой вершины запускается Дейкстра

Назначение:
Кратчайшие пути **между всеми парами вершин** в разреженных графах.

Сложность:

- время: **$O(n \cdot m \log n)$**

Краткое сравнение (строго по лекциям)

Алгоритм	Веса	Отрицательные	Все пары	Сложность
BFS	1	нет	нет	O(n+m)
Дейкстра	≥ 0	нет	нет	O(m log n)
Форд–Беллман	любые	да	нет	O(n·m)
Флойд–Уоршелл	любые	да	да	$O(n^3)$
Джонсон	любые	нет циклов	да	O(n·m log n)

НИЖЕ НА ВСЯКИЙ СЛУЧАЙ ПРИМЕРЫ ВСЕХ АЛГОРИТМОВ НА ПСЕВДОКОДЕ. <u>СДЕЛАНЫ НЕ ПО ЛЕКЦИЯМ</u>

1) BFS (для невзвешенного графа / веса 1)

text

```
bfs(graph, start):
    dist[0..n-1] = INF
    dist[start] = 0
    q = queue()
    q.push(start)

    while q not empty:
        u = q.pop()
        for v in graph[u]:
            if dist[v] == INF:
                dist[v] = dist[u] + 1
                q.push(v)

    return dist
```

2) Дейкстра (неотрицательные веса)

text

```
dijkstra(graph, start):
    dist[0..n-1] = INF
    dist[start] = 0
    heap = min_heap()
    heap.push((0, start))

    while heap not empty:
        (d, u) = heap.pop_min()
        if d != dist[u]:
            continue
        for (v, w) in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heap.push((dist[v], v))
```

```
    return dist
```

3) Форд–Беллман (возможны отрицательные рёбра)

text

```
ford_bellman(edges, n, start):
    dist[0..n-1] = INF
    dist[start] = 0

    repeat (n - 1) times:
        for (u, v, w) in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # проверка отрицательного цикла
    for (u, v, w) in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            error("negative cycle")

    return dist
```

4) Флойд–Уоршелл (все пары вершин)

text

```
floyd_warshall(graph):   # graph = матрица смежности
    dist = copy(graph)

    for k in 0..n-1:
        for i in 0..n-1:
            for j in 0..n-1:
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist
```

5) Джонсон (все пары; отрицательные рёбра допустимы, но без отрицательных циклов)

text

```
johnson(graph, n):
    edges = []
    for u in 0..n-1:
        for (v, w) in graph[u]:
            edges.append((u, v, w))

    # добавляем фиктивную вершину q = n
    for v in 0..n-1:
        edges.append((n, v, 0))

    h = ford_bellman(edges, n + 1, n)   # потенциалы

    new_graph = empty adjacency list for 0..n-1
    for u in 0..n-1:
```

```
        for (v, w) in graph[u]:
            new_w = w + h[u] - h[v]
            new_graph[u].append((v, new_w))

    dist = matrix(n x n, INF)
    for u in 0..n-1:
        dist[u] = dijkstra(new_graph, u)
        for v in 0..n-1:
            if dist[u][v] != INF:
                dist[u][v] = dist[u][v] - (h[u] - h[v])

    return dist
```

Лекции 20-21. Хеширование и фильтр Блума.

38. Что такое хеш-таблица? Какие операции она поддерживает? Опишите метод цепочек и метод открытой адресации. Что такое коэффициент заполнения и перехеширование?

Хеш-таблица

Определение (из файла):
Хеш-таблица — это структура данных, которая хранит элементы по ключу, используя **хеш-функцию**, отображающую ключ в индекс массива.

Поддерживаемые операции

Из файла:

- `insert(key, value)` — вставка
- `find(key)` — поиск
- `delete(key)` — удаление

В среднем все операции выполняются за **O(1)**.

Метод цепочек (chaining)

Идея (из файла):
Каждая ячейка таблицы хранит **список** элементов с одинаковым хешем.

Описание:

- хеш-функция даёт индекс `i`;
- элемент добавляется в список в ячейке `i`;
- при коллизии элементы просто накапливаются в цепочке.

Сложность:

- в среднем: **O(1)**
- в худшем случае (все в одной цепочке): **O(n)**

Метод открытой адресации

Идея (из файла):
Все элементы хранятся **непосредственно в массиве**.
При коллизии ищется другая свободная ячейка по некоторому правилу.

Основные варианты пробирования (перечислены в файле):

- линейное пробирование
- квадратичное пробирование
- двойное хеширование

Особенность:
коэффициент заполнения должен быть **< 1**, иначе вставка невозможна.

Коэффициент заполнения

Определение (из файла):
Коэффициент заполнения

$$\alpha = \frac{\text{число элементов}}{\text{размер таблицы}}$$

Он влияет на:

- длину цепочек (при chaining);
- число проб при открытой адресации.

Перехеширование (rehashing)

Определение (из файла):
Перехеширование — это процесс **увеличения размера таблицы** и повторной вставки всех элементов с новой хеш-функцией (или новым модулем).

Когда выполняется:

- при превышении допустимого коэффициента заполнения.

Цель:
сохранить среднюю сложность операций **O(1)**.

39. Сравните методы разрешения коллизий: метод цепочек и метод открытой адресации.

Сравнение методов разрешения коллизий в хеш-таблицах

Метод цепочек (chaining)

Суть (из файла):
В каждой ячейке таблицы хранится **цепочка (список)** элементов с одинаковым хешем.

Плюсы:

- коэффициент заполнения **может быть > 1**;
- проще реализация удаления;

- меньше проблем при высокой загрузке.

Минусы:

- дополнительная память на списки;
- возможны длинные цепочки (худший случай — $O(n)$).

Метод открытой адресации

Суть (из файла):
Все элементы хранятся **в массиве**; при коллизии выполняется **пробирование** свободных ячеек.

Плюсы:

- компактное хранение (без списков);
- лучшая локальность памяти (быстрее на практике при малом α).

Минусы:

- коэффициент заполнения **должен быть < 1**;
- усложнённое удаление;
- производительность резко падает при росте α.

Итоговое сравнение (из файла)

- **Цепочки** — устойчивее при высокой загрузке, проще в поддержке.
- **Открытая адресация** — быстрее и компактнее при низком коэффициенте заполнения, но чувствительна к коллизиям.

40. Что такое идеальное хеширование? Опишите двухуровневую схему идеального хеширования. Что такое хеширование кукушки? Как выполняются вставка и поиск?

Идеальное хеширование

Определение (из файла):
Идеальное хеширование — это метод хеширования для **статического множества ключей**, при котором **коллизии отсутствуют**, а поиск выполняется за **O(1) в худшем случае**.

Двухуровневая схема идеального хеширования

Идея (из файла):
Используются **две хеш-таблицы** и **две хеш-функции**.

Первый уровень

- выбирается хеш-функция h_1;
- все ключи распределяются по ячейкам основной таблицы;
- в каждой ячейке образуется свой набор ключей.

Второй уровень

- для каждой ячейки с k ключами строится **своя таблица размера k^2**;

- подбирается хеш-функция h_2, при которой **внутри ячейки нет коллизий**;
- если коллизии есть — h_2 выбирается заново.

Свойства (из файла):

- суммарная память — **O(n)**;
- поиск — **O(1) в худшем случае**;
- структура **не поддерживает эффективные вставки** (статическая).

Хеширование кукушки (Cuckoo hashing)

Идея (из файла):
Используются **две хеш-функции** h_1, h_2.
Каждый ключ может храниться **в одной из двух позиций**.

Поиск

Поиск (из файла):
Для ключа x проверяются **две ячейки**:

```
h₁(x) и h₂(x)
```

Если ключ есть — он обязательно в одной из них.

Сложность поиска: O(1) в худшем случае.

Вставка

Вставка (из файла):

1. поместить ключ в позицию $h_1(x)$;
2. если ячейка занята — вытеснить старый ключ;
3. вытесненный ключ попытаться вставить в его альтернативную позицию;
4. процесс повторяется (как «кукушка выталкивает яйца»).

Если возникает **цикл вытеснений**, выполняется **перехеширование** с новыми хеш-функциями.

Сравнение (из файла)

- **Идеальное хеширование**:
  - статическое множество;
  - поиск $O(1)$ в худшем случае;
  - нет вставок.
- **Хеширование кукушки**:
  - динамическое;
  - поиск $O(1)$ в худшем случае;

  - вставка обычно `O(1)`, но возможна перехешировка.

41. Что такое фильтр Блума? Каковы его основные операции? Какие преимущества и недостатки? Как оценивается вероятность ложноположительного срабатывания? От чего она зависит?

Фильтр Блума

Определение (из файла):
Фильтр Блума — это **вероятностная структура данных** для проверки принадлежности элемента множеству, которая допускает **ложноположительные ответы**, но **не допускает ложноотрицательных**.

Основные операции

Из файла:

- **add(x)** — добавить элемент x в фильтр
- **check(x)** — проверить, мог ли элемент x быть добавлен

Операции работают за **O(k)**, где k — число хеш-функций.

Принцип работы

Из файла:

- используется битовый массив размера `m`;
- применяются k независимых хеш-функций;
- при добавлении x все биты $h_1(x)$, …, `h_k(x)` устанавливаются в 1;
- при проверке x:
  - если **хотя бы один бит = 0**, элемента точно **нет**;
  - если **все биты = 1**, элемент **возможно есть**.

Преимущества

Из файла:

- очень **компактное** хранение;
- быстрые операции;
- отсутствие ложноотрицательных ответов.

Недостатки

Из файла:

- возможны **ложноположительные срабатывания**;

- стандартный фильтр **не поддерживает удаление** элементов;
- не позволяет получить сами элементы множества.

Вероятность ложноположительного срабатывания

Оценка (из файла):

Вероятность того, что проверка вернёт «элемент есть», хотя его нет: $p \approx \left(1 - e^{-kn/m}\right)^k$

где:

- n — число добавленных элементов;
- m — размер битового массива;
- k — число хеш-функций.

От чего зависит вероятность ошибки

Из файла:

- от **числа элементов** n;
- от **размера фильтра** m;
- от **числа хеш-функций** k.

При фиксированных n и m существует **оптимальное k**, минимизирующее вероятность ошибки.

Ответы по лекциям закончены. Всё что представленно далее может быть не по ним.

Классы, объекты, инкапсуляция

42. Что такое класс в объектно-ориентированном программировании? Что такое объект и как он связан с классом? Какие элементы могут входить в состав класса?

Важно:
Этого вопроса нет в файле с лекциями (КАСД, алгоритмы и структуры данных).
Ниже даю **обычный теоретический ответ по ООП**, **не из файла**.

Класс в объектно-ориентированном программировании

Класс — это **шаблон (описание, модель)**, который задаёт:

- какие **данные** будут у объектов;
- какие **действия (методы)** они могут выполнять.

Класс сам по себе **не является сущностью программы**, а лишь описывает её.

Объект и связь с классом

Объект — это **экземпляр класса**, конкретная сущность, созданная по этому шаблону.

Связь:

- класс — это **чертёж**;
- объект — **конкретная реализация** этого чертежа в памяти.

Пример (словесно):

класс «Автомобиль» → объект «мой красный Ford с номером …»

Элементы, входящие в состав класса

В класс могут входить:

- **Поля (атрибуты)** — хранят состояние объекта
- **Методы** — описывают поведение объекта
- **Конструкторы** — создают и инициализируют объект
- **Свойства** — управляемый доступ к данным
- **Модификаторы доступа** (`public`, `private`, и т.д.)
- **Вложенные типы** (реже)
- **Статические члены** (относятся ко всему классу, а не к объекту)

43. Что такое конструктор класса? Модификаторы доступа. Принцип инкапсуляции и его реализация в C#. Что такое свойства класса и какие бывают виды свойств, включая автоматические?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **короткий теоретический ответ по ООП и C#**, **не из файла**.

Конструктор класса

Конструктор — это специальный метод класса, который:

- вызывается **автоматически при создании объекта**;
- инициализирует поля объекта;
- имеет **то же имя, что и класс**;
- **не имеет типа возвращаемого значения**.

В C# может быть:

- без параметров;
- с параметрами;
- перегруженным.

Модификаторы доступа (C#)

Определяют, **откуда доступен** класс или его члены:

- `public` — доступен отовсюду

- `private` — доступен только внутри класса
- `protected` — доступен в классе и его наследниках
- `internal` — доступен внутри сборки
- `protected internal` — объединение `protected` и `internal`

Инкапсуляция

Инкапсуляция — принцип ООП, при котором:

- **внутреннее состояние объекта скрыто**;
- доступ к данным осуществляется **через методы или свойства**;
- пользователь класса не зависит от внутренней реализации.

Реализация в C#

- поля объявляются `private`;
- доступ к ним осуществляется через **свойства** или методы;
- используется контроль доступа (`get`, `set`).

Свойства класса

Свойство — это элемент класса, который:

- выглядит как поле;
- но работает как методы (`get` / `set`);
- используется для **контролируемого доступа к данным**.

Виды свойств

1. **Обычное свойство**

csharp

```
private int x;
public int X
{
    get { return x; }
    set { x = value; }
}
```

2. **Автоматическое свойство**

csharp

```
public int X { get; set; }
```

- компилятор сам создаёт скрытое поле;
- удобно, когда не нужна логика в `get/set`.

3. **Только для чтения**

csharp

```
public int X { get; }
```

4. **Только для записи**

csharp

```
public int X { set { ... } }
```

5. **С модификаторами доступа**

csharp

```
public int X { get; private set; }
```

44. Что такое статические члены класса и статический конструктор? Как с помощью ключевого слова this можно обратиться к текущему экземпляру класса?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Статические члены класса

Статические члены — это члены класса, которые:

- принадлежат **самому классу**, а не объекту;
- существуют в **единственном экземпляре**;
- доступны **без создания объекта**.

В C# объявляются с ключевым словом `static`.

К статическим членам относятся:

- статические поля;
- статические методы;
- статические свойства.

Пример (словесно):

счётчик объектов, общая константа, вспомогательная функция.

Статический конструктор

Статический конструктор:

- инициализирует **статические поля класса**;
- вызывается **автоматически один раз**;
- выполняется **до первого использования класса**;
- **не имеет параметров** и **модификаторов доступа**.

В классе может быть **только один** статический конструктор.

Ключевое слово `this`

`this` — ссылка на **текущий экземпляр класса**.

Используется для:

1. обращения к полям и методам текущего объекта;
2. устранения конфликта имён (параметр = поле);
3. вызова другого конструктора того же класса.

Пример (словесно):

`this.x` — поле объекта,
`this.Method()` — метод текущего объекта.

 В **статических методах** `this` **недоступно**, так как нет экземпляра класса.

Наследование

45. Что такое наследование? Как в C# обозначается наследование? Какие члены наследуются? Что такое запечатанный класс? Какие ограничения на наследование существуют?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по ООП и C#, не из файла**.

Наследование

Наследование — это механизм ООП, при котором **один класс (производный)** получает поля и методы **другого класса (базового)** и может:

- использовать их;
- расширять;
- переопределять поведение.

Цель — **повторное использование кода** и логическое обобщение.

Как обозначается наследование в C#

В C# наследование задаётся **через двоеточие `:`**:

csharp

```
class Derived : Base
{
}
```

Какие члены наследуются

В производном классе наследуются:

- `public` — наследуются и доступны
- `protected` — наследуются и доступны в наследнике
- `internal` — наследуются (в пределах сборки)
- `protected internal` — наследуются

 Не наследуются:

- `private` члены базового класса
- конструкторы базового класса (но они вызываются)

Запечатанный класс (`sealed`)

Запечатанный класс — это класс, **от которого нельзя наследоваться**.

В C# помечается ключевым словом:

csharp

```
sealed class MyClass
{
}
```

Назначение:

- запретить расширение класса;
- повысить безопасность;
- иногда — оптимизация.

Ограничения на наследование в C#

1. **Нет множественного наследования классов**

   csharp

   ```
   class A : B, C ✕
   ```

   Разрешено множественное **наследование интерфейсов**.

2. Нельзя наследоваться от `sealed` класса
3. Производный класс **не имеет доступа к `private` членам** базового
4. Переопределять можно **только `virtual` / `abstract` члены** (через `override`)
5. Базовый класс должен быть **доступен** (по модификатору доступа)

46. В каком случае производный класс должен вызывать конструктор базового класса? Как с помощью ключевого слова base можно обратиться к базовому классу?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Когда производный класс должен вызывать конструктор базового класса

Производный класс **обязан вызвать конструктор базового класса**, если:

- у базового класса **нет конструктора без параметров**;
- базовый класс требует инициализации данных через параметры;
- необходимо явно выбрать **какой именно конструктор базового класса** использовать.

Вызов конструктора базового класса **происходит всегда** —
либо **явно**, либо **неявно**.

Как вызывается конструктор базового класса

В C# вызов выполняется **в списке инициализации конструктора**
с помощью ключевого слова `base`:

csharp

```
class Derived : Base
{
    public Derived(int x) : base(x)
    {
    }
}
```

Ключевое слово `base`

`base` — это ссылка на **базовый класс** текущего объекта.

С помощью `base` можно:

1. **Вызвать конструктор базового класса**

csharp

```
base(arg1, arg2)
```

2. **Обратиться к полям и методам базового класса**

csharp

```
base.Method();
base.Field;
```

Используется, когда:

- член базового класса **скрыт** в производном;
- нужно явно вызвать реализацию из базового класса.

Ограничения

- `base` можно использовать **только в нестатическом контексте**;
- вызов `base(...)` возможен **только в конструкторе** и должен быть **первым**.

Полиморфизм

47. Что такое полиморфизм? Что такое виртуальный метод и модификатор virtual? Чем override отличается от new? Что такое абстрактный класс и абстрактный метод? Можно ли создать экземпляр абстрактного класса?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по ООП и C#, не из файла**.

Полиморфизм

Полиморфизм — это способность объектов **разных классов** обрабатываться **через общий базовый тип**, при этом вызываться будет **реализация конкретного объекта**.

Идея: *один интерфейс — разное поведение*.

Виртуальный метод и `virtual`

Виртуальный метод — метод базового класса, который **может быть переопределён** в производном классе.
В C# объявляется с ключевым словом `virtual`.

csharp

```
class Base
{
    public virtual void Foo() { }
}
```

`override` и `new` — в чём разница

`override`

- **переопределяет** виртуальный метод базового класса;
- работает через **полиморфизм**;
- выбор метода зависит от **реального типа объекта**.

csharp

```
class Derived : Base
{
    public override void Foo() { }
}
```

`new`

- **скрывает** метод базового класса;
- **не является полиморфизмом**;

- выбор метода зависит от **типа ссылки**.

csharp

```
class Derived : Base
{
    public new void Foo() { }
}
```

Ключевая разница:

- `override` — правильное переопределение;
- `new` — сокрытие метода.

Абстрактный класс

Абстрактный класс — это класс:

- предназначенный для **наследования**;
- может содержать **реализацию и абстрактные методы**;
- объявляется с `abstract`.

csharp

```
abstract class Shape
{
}
```

Абстрактный метод

Абстрактный метод:

- **не имеет реализации**;
- **обязан быть реализован** в неабстрактном наследнике;
- объявляется с `abstract`.

csharp

```
abstract class Shape
{
    public abstract double Area();
}
```

Можно ли создать экземпляр абстрактного класса?

Нет, нельзя.

Абстрактный класс используется **только как базовый** для других классов.

48. Что такое перегрузка методов? Что такое анонимные типы и где применяются?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Перегрузка методов (method overloading)

Перегрузка методов — это объявление **нескольких методов с одинаковым именем**, но **разными сигнатурами** (набором параметров).

Различаться могут:

- количество параметров;
- типы параметров;
- порядок параметров.

 Нельзя перегружать метод **только по возвращаемому типу**.

Зачем нужна:

- удобство и читаемость API;
- логически одинаковые операции с разными входными данными.

Анонимные типы

Анонимный тип — это тип **без явного имени**, создаваемый «на лету» с набором свойств.

В C# создаётся с помощью `new { ... }`.

Особенности:

- свойства **только для чтения**;
- тип доступен **только в текущем контексте**;
- компилятор сам генерирует класс.

Где применяются анонимные типы

Основные случаи использования:

- **LINQ-запросы** (проекции данных);
- временные структуры для группировки данных;
- передача нескольких значений без объявления отдельного класса.

Типичный сценарий:

выбрать несколько полей из объекта и работать с ними как с единым значением.

Интерфейсы

49. Что такое интерфейс и чем он отличается от абстрактного класса? Сколько интерфейсов может реализовать класс? Наследование интерфейсов. Все ли члены интерфейса должны быть реализованы? Какие модификаторы по умолчанию?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Интерфейс

Интерфейс — это контракт, который задаёт **набор членов**, которые класс **обязан реализовать**, не предоставляя реализации (классически).

В C# объявляется ключевым словом `interface`.

Чем интерфейс отличается от абстрактного класса

Интерфейс:

- описывает *что* класс должен уметь;
- **не хранит состояние** (полей экземпляра);
- **не содержит конструкторов**;
- класс может реализовать **несколько интерфейсов**.

Абстрактный класс:

- может содержать **реализацию и поля**;
- может иметь конструкторы;
- поддерживает **только одиночное наследование**.

Сколько интерфейсов может реализовать класс

Сколько угодно.
В C# разрешено **множественное наследование интерфейсов**:

csharp

```
class MyClass : IFirst, ISecond, IThird
{
}
```

Наследование интерфейсов

Интерфейс может **наследоваться от других интерфейсов**:

csharp

```
interface IA { }
interface IB { }
interface IC : IA, IB { }
```

Класс, реализующий `IC`, обязан реализовать **все члены всех базовых интерфейсов**.

Все ли члены интерфейса должны быть реализованы

- **Да**, если класс **не абстрактный**
- **Нет**, если класс объявлен `abstract`

Начиная с **C# 8**, интерфейсы **могут содержать реализации по умолчанию**, но:

- класс **не обязан** их переопределять;
- это исключение, а не классическое правило.

Модификаторы по умолчанию

В интерфейсе:

- все члены по умолчанию **`public`**;
- указывать `public` **необязательно**;
- `private`, `protected` (классически) **недопустимы**
  (кроме новых возможностей C# 8+ для служебных методов).

Делегаты и события

50. Что такое делегат? Что такое многоадресный делегат? Что такое событие и чем оно отличается от делегата? Что такое анонимный метод? Каков стандартный шаблон EventArgs?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Делегат

Делегат — это тип, который хранит **ссылку на метод** с определённой сигнатурой и позволяет вызывать этот метод как функцию.

Используется для:

- передачи методов как параметров;
- реализации колбэков;
- событийной модели.

Многоадресный делегат

Многоадресный делегат — делегат, который содержит **список методов** (invocation list).

- при вызове делегата вызываются **все методы по порядку**;
- возвращаемое значение (если есть) — от **последнего метода**.

Событие (`event`)

Событие — это механизм уведомления, построенный **на основе делегата**, но с **ограниченным доступом**.

Отличие события от делегата

- делегат можно **присвоить извне**;
- событие — **нельзя вызвать или перезаписать извне**, можно только:
  - подписаться (+=);
  - отписаться (-=).

Идея:
делегат — механизм,
событие — **безопасная обёртка** для уведомлений.

Анонимный метод

Анонимный метод — метод **без имени**, объявляемый прямо в месте использования делегата.

Используется для:

- краткой логики;
- обработчиков событий;
- лямбда-выражений (современная форма).

Стандартный шаблон `EventArgs`

В C# принят стандартный шаблон событий:

csharp

```
void Handler(object sender, EventArgs e)
```

Где:

- `sender` — объект-источник события;
- `EventArgs` — данные события.

Если нужны данные:

csharp

```
class MyEventArgs : EventArgs
{
    public int Value { get; }
}
```

Исключения

51. Что такое исключение? Для чего нужна обработка исключений? Как работают try-catch-finally? Всегда ли выполняется finally? Как сгенерировать исключение оператором throw? Можно ли создавать собственные типы исключений?

Важно:
Этого вопроса нет в файле с лекциями (КАСД).
Ниже — **краткий корректный ответ по C# / ООП**, **не из файла**.

Исключение

Исключение — это объект, описывающий **ошибку во время выполнения программы**, которая нарушает нормальный ход выполнения.

Зачем нужна обработка исключений

Обработка исключений позволяет:

- **перехватывать ошибки** без аварийного завершения программы;
- корректно **освобождать ресурсы**;
- отделять **логику обработки ошибок** от основной логики.

`try – catch – finally`

csharp

```
try
{
```

```
    // код, где может возникнуть исключение
}
catch (ExceptionType e)
{
    // обработка исключения
}
finally
{
    // код очистки ресурсов
}
```

- `try` — код с потенциальной ошибкой
- `catch` — перехват и обработка исключения
- `finally` — выполняется для завершения/очистки

Всегда ли выполняется `finally`?

Почти всегда:

- выполняется при нормальном завершении `try`;
- выполняется при возникновении исключения;
- выполняется даже при `return` из `try` или `catch`.

Может **не выполниться**, если:

- процесс аварийно завершён;
- вызван `Environment.FailFast`.

Генерация исключения (`throw`)

Исключение можно **явно выбросить** оператором `throw`:

csharp

```
throw new Exception("Ошибка");
```

Также можно **пробросить текущее исключение**:

csharp

```
throw;
```

Пользовательские (собственные) исключения

Да, можно создавать.

Обычно наследуются от `Exception`:

csharp

```
class MyException : Exception
{
    public MyException(string message) : base(message) { }
}
```

Используются для:

- описания **доменных ошибок**;
- более точной обработки ошибок.